

Day 1

Course Philosophy

This is a **reverse-engineered** course — it teaches exactly what gets asked in AI interviews, nothing wasteful. It avoids two bad extremes:

- **Too math-heavy** (transformer math, backprop, PyTorch internals) → good for PhD/research, not interviews.
- **Wrapper tutorials** (“build a chatbot in 10 min”) → feels productive, teaches nothing. You can’t answer a single “why” in an interview.

Tip: In interviews, code is never asked. You’re asked “*what is this code actually doing?*” Understand concepts; don’t memorize code.

Who it’s for: college students, working professionals (1–3 yrs) switching to AI, service-based folks wanting into AI projects.

Not for: anyone wanting AI/ML math or a PhD track.

Prerequisites: a laptop, internet, a notebook, and basic Python (variables, lists, tuples, dictionaries).

What You’ll Install (Day 1 setup)

1. **Python**
2. **VS Code** + 3 extensions: **Python**, **Pylance**, **Jupyter** (all by Microsoft)
3. **Git** + a **GitHub** account
4. **Groq** cloud account → generate an **API key**
5. **Qdrant** account (vector DB, used later for RAG) → API key + cluster endpoint
6. **uv** (a fast Python package manager, like pip but faster)

Mac note (installation):

- **Python:** Download the **macOS installer** from python.org (not the Windows one). Or simpler — `brew install python` if you have Homebrew.
- **VS Code:** Download the **Mac (Apple Silicon / Intel)** build. Drag it into your `Applications` folder.
- **Git:** Usually already on Mac. Check with `git --version` (it may prompt to install Xcode Command Line Tools — allow it).
- **uv:** `curl -LsSf https://astral.sh/uv/install.sh | sh`

- **Terminal:** No need for “Microsoft Terminal”. Use the built-in **Terminal** app (Cmd + Space → type “Terminal”). iTerm2 is a nice optional upgrade.
- Wherever the video presses `Windows + R`, you just open **Terminal** on Mac.

Set up Git identity (same on Mac):

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
git config --global --list          # verify
```

Store secrets in a `.env` file (never commit to Git):

```
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXX
QDRANT_API_KEY=XXXXXXXXXXXXXXXXX
QDRANT_URL=https://xxxx.qdrant.io
```

Tip: The API key is shown **only once**. Copy it immediately and save it somewhere safe. If lost, you must regenerate.

Virtual Environment (venv) — why it matters

Not an AI thing — a general software practice.

GTA analogy: GTA5 needs driver v2.0, GTA6 needs v4.0. If you upgrade the whole laptop’s driver for GTA6, GTA5 breaks. Each game needs its own isolated “room”.

In code: Week 1’s project needs one package version; Week 2 needs a newer one. Upgrade globally and the old project breaks. A venv gives each project its own isolated Python + packages.

```
uv init day1          # creates the day1 folder
cd day1
uv venv --python 3.11  # create the virtual environment
source .venv/bin/activate # 🍏 MAC activate command
python3 --version      # verify
```

🍏 Mac note:

- Activate differs from Windows. Windows: `.venv\Scripts\activate` → **Mac:** `source .venv/bin/activate`
- Use `python3` (and `pip3`) on Mac, not `python`. Better yet, use `uv run` and skip the confusion entirely.

- Deactivate is the same: `deactivate`
- Copy the `.env` into each day's folder. On Mac: `cp ../.env .` (or drag it in Finder). No copy "command" needed if you use Finder.

Tip: When you see `(day1)` before your prompt, you're inside the `venv`.

Verify your setup (final checks)

```
python3 --version
uv --version
git --version
code --version          # VS Code (needs "shell command" installed once from
VS Code)
```

 **Mac note:** To enable the `code` command on Mac: open VS Code → `Cmd + Shift + P` → type “**Shell Command: Install ‘code’ command in PATH**” → hit enter. Now `code .` works from Terminal.